



Payments Integration Guide

Prepared for NurseIO

Low-friction contractor payout onboarding, embedded in the NurseIO app.

Overview

NurseIO already collects and verifies a nurse's identity and tax information in your own onboarding, before the nurse ever reaches payment setup. This guide shows how to hand that data to Wingspan server-side, certify the nurse's W-9 on their behalf, and then embed a single Wingspan screen, the payout chooser, inside the NurseIO app.

The model is payer-driven. Your backend does the work through the Wingspan API: it creates the contractor, submits identity and tax, runs verification, and records the W-9 certification. The only thing the nurse touches is picking how they get paid, shown in a slide-up panel inside NurseIO via the Wingspan embedded SDK. If you default them to the Wingspan Wallet, even that step can disappear.

Everything in this guide that runs against the API has been verified end-to-end on NurseIO's staging tenant.

Environment and authentication

Surface	Staging	Production
API base	<code>https://stagingapi.wingspan.app</code>	<code>https://api.wingspan.app</code>
Embed host (SDK)	<code>https://staging-embedded.wingspan.app</code>	<code>https://embedded.wingspan.app</code>

Server-side (pre-provision) auth: your NurseIO parent API token as `Authorization: Bearer {PARENT_TOKEN}`, plus an `X-WINGSPAN-USER: {accountId}` header to act as a specific account, your payee bucket when creating a contractor, or the nurse herself for the onboarding calls. The `payeeId` you get back from creating the contractor is the nurse's user id, and it is the value you pass as `X-WINGSPAN-USER` for every `/v2/onboarding/*` call.

Embed auth: the SDK takes a short-lived per-nurse session token and hands it to the iframe over a `postMessage` handshake. The token never appears in the URL, and your parent API token never reaches the browser.

How it works

1. Your backend pre-provisions the nurse: create, submit identity + tax, verify, certify the W-9 (Part A for an individual, Part B for a business).
 2. Your backend mints a session token for that nurse.
 3. Your app opens the Wingspan embedded SDK in a slide-up and shows just the payout step (Part C).
 4. The nurse picks a payout method, the SDK fires `onComplete`, and you route them on.
- Because steps 1-2 happen server-side, the nurse never sees identity, tax, or W-9 screens.

Part A: Pre-provision an individual nurse

Every call uses your parent token; `X-WINGSPAN-USER` sets the context.

Step 0: Create the contractor

```
POST {API}/payments/payee
Authorization: Bearer {PARENT_TOKEN}
X-WINGSPAN-USER: {PAYEE_BUCKET_ID}
{ "email": "nurse@example.com", "firstName": "Jane", "lastName": "Smith",
  "payerOwnedData": { "payeeExternalId": "NIO-123", "status": "Active" } }
```

Returns the nurse's `payeeId` at the top level. That value is the nurse's user id and your `X-WINGSPAN-USER` for every step below. (It is not `user.userId`, which is not returned.)

Step 1: Create the customer entity

```
POST {API}/v2/onboarding/customer
X-WINGSPAN-USER: {payeeId}
{ "type": "Individual", "country": "US" }
```

Creates the record the verification system reads. Do this before the next calls, or they return "no customer entity."

Step 2: Submit identity and tax (including SSN)

```
PATCH {API}/v2/onboarding/customer/Entity
X-WINGSPAN-USER: {payeeId}
{ "customerData": {
  "firstName": "Jane", "lastName": "Smith", "occupation": "Registered Nurse",
  "dateOfBirth": "1991-04-22", "country": "US", "individualTaxId": "123456789",
  "addressLine1": "1500 N Central Ave", "city": "Phoenix", "region": "AZ",
  "postalCode": "85004", "email": "nurse@example.com", "phoneNumber":
    "+16025550199" } }
```

Step 2b: Validate (recommended)

```
GET {API}/v2/onboarding/missing-data/Tax      X-WINGSPAN-USER: {payeeId}
GET {API}/v2/onboarding/missing-data/Banking X-WINGSPAN-USER: {payeeId}
```

The backend decides what each level needs, and Banking (the Wingspan Wallet) can require more than Tax. Fill whatever `requiredFields` lists, then re-check until both come back empty. Do not rely on a fixed field list; let this endpoint tell you.

Step 3: Record the banking acknowledgements

Only needed if the nurse will use the Wingspan Wallet (or the Wallet card). Each is a `POST` to `/v2/onboarding/acknowledgements/{name}`:

```
POST {API}/v2/onboarding/acknowledgements/DepositAccountHolderAgreement
X-WINGSPAN-USER: {payeeId}
{ "acknowledgementName": "DepositAccountHolderAgreement", "acknowledgementStatus":
  "Given", "version": "2024-07-22" }
```

Send the three banking acknowledgements (`DepositAccountHolderAgreement`, `LeadBankTerms`, `ElectronicDisclosureAndConsent`), plus `WingspanTosAcceptance` and `WingspanPrivacyPolicyAcceptance` if they were not already accepted. See the [Acknowledgements](#) reference for names and versions.

Step 4: Run verification, then poll

```
POST {API}/v2/onboarding/verifications/Tax      X-WINGSPAN-USER: {payeeId}
POST {API}/v2/onboarding/verifications/Banking  X-WINGSPAN-USER: {payeeId}
GET  {API}/v2/onboarding/verifications          X-WINGSPAN-USER: {payeeId}
```

Poll the `GET` until `tax` and `banking` read `Verified`. In production these run for real (the IRS TIN check, and a banking identity check for the Wallet) and are asynchronous, so poll rather than reading once. A `Failed` or `UpdateRequired` result returns a token for additional data collection that cannot always be resolved server to server. In staging the checks auto-pass once the data is complete.

Step 5: Certify the W-9

```
POST {API}/v2/onboarding/acknowledgements/W9Certification
X-WINGSPAN-USER: {payeeId}
{ "acknowledgementName": "W9Certification", "acknowledgementStatus": "Given",
  "version": "2024-03-01" }
```

This is the actual W-9 certification, done on the nurse's behalf. (Non-US contractors use `W8BenCertification`.)

Step 6: Consent to electronic 1099 delivery

```
POST {API}/v2/onboarding/acknowledgements/ElectronicTaxFormConsent
X-WINGSPAN-USER: {payeeId}
{ "acknowledgementName": "ElectronicTaxFormConsent", "acknowledgementStatus":
  "Given", "version": "2024-08-01" }
```

Step 7: Authorize sharing the W-9 with the payer

```
PATCH {API}/payments/payer/{PAYER_ID}
X-WINGSPAN-USER: {payeeId}
{ "payeeOwnedData": { "shareTaxDocument": "Allow" } }
```

`{PAYER_ID}` is a payer the nurse is actually related to. The relationship is created when you create the payee in that payer's context, so the bucket works here; if you also pay through a separate EIN entity, attach the nurse to it first, then set this for that payer too.

After Part A the nurse is identity-verified, tax-verified, and W-9-certified, with nothing for them to fill. Skip to Part C.

Part B: Pre-provision a business contractor

Some nurses operate as an LLC or other entity. The chain is the same as Part A with two differences: you declare the entity as a business, and you seed two records, the business entity and its authorized representative (the human, whose SSN drives the identity check). Steps 0, 2b, 3-7 are identical to Part A.

Step 1: Create the customer entity as a business

```
POST {API}/v2/onboarding/customer
X-WINGSPAN-USER: {payeeId}
{ "type": "Business", "country": "US" }
```

Step 2a: Seed the business entity

```
PATCH {API}/v2/onboarding/customer/Entity
X-WINGSPAN-USER: {payeeId}
{ "customerData": {
  "legalBusinessName": "Smith Nursing LLC",
  "businessTaxId": "123456789",
  "federalTaxClassification": "LlcSingleMember",
  "regionOfFormation": "AZ", "yearOfFormation": "2020",
  "addressLine1": "1500 N Central Ave", "city": "Phoenix", "region": "AZ",
  "postalCode": "85004", "country": "US",
  "phoneNumber": "+16025550199", "email": "biz@example.com",
  "website": "https://smithnursing.example.com",
  "industry": "HealthCareAndSocialAssistance" } }
```

`businessTaxId` is the EIN. `federalTaxClassification` is one of: `SoleProprietorship`, `LlcSingleMember`, `CorporationS`, `CorporationC`, `Partnership`, `LlcCorporationS`, `LlcCorporationC`, `LlcPartnership`, `NotForProfitOrganization`.

Step 2b: Seed the authorized representative

```
PATCH {API}/v2/onboarding/customer/Representative
X-WINGSPAN-USER: {payeeId}
{ "customerData": {
  "firstName": "Jane", "lastName": "Smith", "dateOfBirth": "1991-04-22",
  "individualTaxId": "111223333", "ownershipPercent": "100",
  "occupation": "Registered Nurse", "phoneNumber": "+16025550199",
  "email": "jane@example.com",
  "addressLine1": "742 Evergreen Ter", "city": "Tempe", "region": "AZ",
  "postalCode": "85281", "country": "US" } }
```

`individualTaxId` here is the representative's SSN. Then run the validate step (missing-data/Tax and /Banking); a business commonly still needs the business `website` and the representative's `phoneNumber` and `email`, so check and fill before verifying. From here, steps 3-7 (acknowledgements, verify and poll, W-9 certify, 1099 consent, share) are exactly as in Part A. The W-9 certification is still `W9Certification`; the representative certifies on the entity's behalf.

Part C: Embed the payout step

Step 1: Install

Install from npm (public package, no auth, registry config, or .npmrc needed):

```
npm i @wingspan/embedded-sdk
```

Or skip the bundler entirely and load it from a CDN; the script-tag build defines Wingspan on window:

```
<script
  src="https://unpkg.com/@wingspan/embedded-sdk@1.0.0/dist/index.global.js"></script>
```

Requires Node 18 or newer for the npm path. Pin a version, in the install or the CDN URL, for production, so a new release cannot change under you. The unversioned CDN URL resolves to the latest published version.

Step 2: Expose a session token from your backend

Mint the per-nurse token server-side and hand it to your frontend. Your parent token stays on the server.

```
GET {API}/users/organization/user/{payeeId}/session
Authorization: Bearer {PARENT_TOKEN}
-> { "token": "{session-token}", "userId": "{payeeId}" }
```

Step 3: Open the payout component in your slide-up

```
const { token } = await fetch("/your-api/wingspan-session").then(r => r.json());
const { Wingspan } = await import("@wingspan/embedded-sdk");

const ws = Wingspan.init({ baseUrl: "https://embedded.wingspan.app", token });

const payout = ws.createManagePayoutMethod({
  allowedUsMethods: ["ach", "wallet", "instant"],
  onComplete: ({ changed }) => {
    if (changed) { /* mark payout set on your side */ }
    advanceNurseToNextScreen(); // you control the redirect
  },
  onError: (e) => reportError(e),
});

payout.openModal({ width: "480px", height: "720px" }); // or
payout.mount("#container")
```

`ManagePayoutMethod` handles both the first method and any later change, so you use the same component for "set up payouts" and "change how I get paid." It fires `complete` with `{ changed }` (true means they saved a method), the modal closes, and you route the nurse onward. There is no fixed redirect URL to configure; you decide the destination in `onComplete`.

Acknowledgements reference

Body shape for every acknowledgement: { "acknowledgementName": "{name}",
 "acknowledgementStatus": "Given", "version": "{date}" }, posted to
 /v2/onboarding/acknowledgements/{name}.

Acknowledgement	Version	When
W9Certification	2024-03-01	W-9 certification (US); W8BenCertification (2021-10-01) for non-US
ElectronicTaxFormConsent	2024-08-01	Electronic 1099 delivery consent
DepositAccountHolderAgreement	2024-07-22	Wallet (banking)
LeadBankTerms	2022-08-09	Wallet (banking)
ElectronicDisclosureAndConsent	2024-08-01	Wallet (banking)
WingspanTosAcceptance	2024-07-22	If not already accepted
WingspanPrivacyPolicyAcceptance	2024-07-22	If not already accepted
DebitCardHolderAgreement	2022-07-22	Wingspan Wallet Visa card
CashBackPromotionAgreement	2022-10-04	Wingspan Wallet Visa card

These version dates change when the underlying terms update, and there is no endpoint to fetch the current version. Post the then-current value or the acknowledgement will not satisfy. We will flag you when any version changes.

The Wingspan Wallet card

Two separate things. Selecting `wallet` in the payout component sets the Wingspan Wallet as the nurse's payout destination, that is covered by Part C. Issuing the Visa debit card itself (so the nurse never sets it up) is a separate server-side capability; we are confirming the API access on our side and will document it once it is enabled for your org. The card acknowledgements (`DebitCardHolderAgreement`, `CashBackPromotionAgreement`) are listed above.

Notes

- Mint the session token server-side; keep your API token out of the browser.
- `payeeId` from Step 0 is the nurse's user id and the `X-WINGSPAN-USER` for all `/v2/onboarding/*` calls. The separate `collaboratorId` (from the payee response `requirements`) is for payables only.
- Staging auto-verifies once data is complete; production runs the real checks asynchronously, so poll.
- `shareTaxDocument` needs an existing payer-payee relationship, set it per payer the nurse is related to.